



Course Code	CSA0315	Course Title	Data Structures
CO Assessed	CO1	Assessment	Assessment Tool 2: Technical Presentation
Weightage	30% of CIA	Total Marks	100
CO1 Statement	Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)		
Student Name	R. NANDHINI	Reg. No.	192524355
Section/Batch	Slot D (2025)	Date	06-10-2026

Code of Conduct:

I **R.Nandhini** , certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Nandhini

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Concept Identification	30%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts	
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established	
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure	
Integration of Literature	15%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence	
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand	

Q-24) A Banking System retrieve Customers Account Details Using Account Numbers Instantly Suggest An Appropriate Data Structures And Justify Your Selection

ASSESSMENT TOOL - 2 TECHNICAL PRESENTATION

BANKING SYSTEM: FAST CUSTOMER ACCOUNT RETRIEVAL

SUMBITTED BY
R.NANDHINI

SUMBITTED TO
Dr. KRISHNARAJ MONY

PROBLEM STATEMENT

Question:

A banking system retrieves customer account details using account numbers instantly. Suggest an appropriate data structure and justify your selection.

Problem:

Banks store a large number of customer accounts.

Every customer has a unique **account number**.

Account details must be retrieved quickly.

Searching through every customer record would take more time.

Therefore, an efficient data structure is required.

PROPOSED DATA STRUCTURE: HASH TABLE

Hash Table

A **Hash Table** stores data in the form of **key-value pairs**.

Key → **Account Number**

Value → **Customer Account Details**

A **hash function** converts the account number into a table index.

The system uses this index to find the required record.

Example:

Account Number



Hash Function



Table Index



Customer Details

WORKING OF HASH TABLE

The retrieval process works in the following steps:

Step 1: Customer enters the account number.

Step 2: The account number is given to the hash function.

Step 3: The hash function calculates a table index.

Step 4: The system goes directly to that index.

Step 5: Customer details are retrieved

User enters Account No. 10025



Hash Function



Index = 5



Hash Table



Customer Details

WHY HASH TABLE IS SUITABLE?

Advantages:

Fast Retrieval: Account details can be accessed very quickly.

Average $O(1)$ Search: Searching normally takes constant time.

Unique Key: Account number can act as a unique key.

Large Data Handling: Suitable for large numbers of accounts.

Efficient Operations: Supports fast insertion, searching and deletion.

Direct Access: The hash function helps locate the record directly.

Main Reason

Account Number $\rightarrow$ Hash Function $\rightarrow$ Direct Location $\rightarrow$ Account Details

TIME AND SPACE COMPLEXITY

Operation	Average Case	Worst Case
Search	$O(1)$	$O(n)$
Insertion	$O(1)$	$O(n)$
Deletion	$O(1)$	$O(n)$

Space Complexity $O(n)$

Where n = number of customer records.

Why can the worst case become $O(n)$?

If multiple account numbers produce the same index, a **collision** occurs. Too many collisions can increase the search time.

Therefore, a good **hash function** and suitable collision-handling technique are important.

COLLISION HANDLING

What is a Collision?

A collision occurs when **two different account numbers generate the same hash-table index**.

Collision Handling Methods

1. Chaining

Multiple records are stored at the same index using a linked list.

2. Open Addressing

If an index is occupied, another empty position is searched.

Common methods:

Linear Probing

Quadratic Probing

Double Hashing

Example:

Account 1001 → Hash Function → Index 5

Account 1021 → Hash Function → Index 5

↑

Collision

CONCLUSION

- A **Hash Table** is an appropriate data structure for banking account retrieval.
- The **account number acts as the key**.
- Customer information is stored as the corresponding value.
- Hashing provides **average $O(1)$ retrieval time**.
- It is efficient for handling large customer databases.
- Collision-handling techniques improve its reliability.
- Therefore, a hash table provides **fast and efficient account lookup**.